CS 8803-O08: Intermediate Representation Specification

Santosh Pande
santosh.pande@cc.gatech.edu

Georgia Institute of Technology — Updated: June 4, 2023

Overview

This document describes the intermediate representation (IR) used for the Tiger Compiler project. The IR is the sole method used to transport information about the source program from the front end to the back end of the compiler. Therefore, the IR must contain all of the information needed by the back end, including names, types, and data operations.

1 IR Form

The IR is a linear 3-address code (often called quadruples when including the operand) which has the following form:

$$op, x, y, z$$

This reads as, "Do operation op to the values x and y. Then, assign the new value to z."

A simple IR example is:

$$2 * a + (b - 3)$$

The IR for this expression is:

```
sub, b, 3, _t1
mult, 2, a, _t2
add _t1, _t2, _t3
```

2 Opcodes

Operation	Source Code	IR
	Assignment	
assign	a := b	assign, a, b,
	Binary Operations	
add	a + b	add, a, b, _t1
sub	a - b	sub, a, b, _t1
mult	a * b	mult, a, b, _t1
div	a / b	div, a, b, _t1
and	a & b	and, a, b, _t1
or	a | b	or, a, b, _t1
	Jump	
goto	break;	goto, after_loop, ,
	Branch	
breq	branch to _Label if a == b	breq, a, b, _Label
brneq	branch to _Label if a != b	brneq, a, b, _Label
brlt	branch to _Label if a < b	brlt, a, b, _Label
brgt	branch to _Label if a > b	brgt, a, b, _Label
brleq	branch to _Label if a <= b	brleq, a, b, _Label
brgeq	branch to _Label if a >= b	brgeq, a, b, _Label
	Return	
return	return a;	return, a, ,
return	return;	return, , ,
	Function Calls	
call	foo(x);	call, foo, x
callr	a := foo(x, y, z);	callr, a, foo, x, y, z
	Array Operations	
array_store	arr[0] := a;	array_store, arr, 0, a
array_load	a := arr[0];	array_load, a, arr, 0
	type ArrayInt = array[100] of int;	
	var X : ArrayInt;	
assign	var Y : ArrayInt := 10;	assign, X, 100, 10
	X := Y	assign, X, Y,

3 Temporary Variables

An important characteristic of IR code is the introduction of temporary variables, which the compiler must generate. The temporary variables must be added to the symbol table and their type should be derived from the result of the operation. In the example in the previous section, `_t1`, `_t2`, and `_t3` would all be marked as type `int`. Types for temporary variables *propagate* through the program. When creating the IR, you may assume infinitely many temporary variables are available.

4 Names

- case-sensitive
- must begin with a letter or underscore
- can be comprised of letters, digits, and underscores
- must be unique

4.1 Name Mangling

All names appearing in IR must be unique throughout the entire program, even across functions. Therefore, names must be altered by the compiler. This renaming is referred to as "name mangling".

One way to account for uniqueness in a variable name is by mangling the variable's name with its scope. To avoid collisions, an underscore "_" prefix should be prepended to names, including auto-generated temporaries and labels.

Following this scheme, if you have two variables named `num`, one in scope 2 and the other in scope 3, they would be renamed to `_2_num` and `_3_num` respectively.

5 Branching

The idea with branches is to skip over a block of code if the logical test is false. If you want to execute code when `a == b`, then you want to jump over that code if `a != b`. For `printi(two == one)` you could generate the following IR:

```
assign, _t1, 0,
brneq, two, one, _if_label_0
assign, _t1, 1,
_if_label_0:
call, printi, _t1
```

6 IR Output File

For this project, the generated IR will be written to a file. The IR file will contain a program header followed by a series of function definitions.

- The program header contains the program name and list of global static variables.
- Each IR function definition includes a function signature, a list of local stack variables, and the opcode instructions.
- A Tiger procedure with no explicit return value should generate IR with return type `void`, and must end with a `return` opcode.
- Static initialization of variables is done at the start of `main`.
- Comments can be included in the IR file by starting the line with a `#` symbol.

Refer to the example programs for the correct output formatting.

7 Example Programs

demo_print.ir

```
start_program demo_print
    static-int-list:
    static-float-list:

start_function printout
    void printout(int _1_x, int _1_y, int _1_z)
    int-list: _1_x, _1_y, _1_z
    float-list:
    printout:
        call, printi, _1_x
        call, printi, _1_y
        call, printi, _1_z
        return, , ,
end_function printout

start_function main
    int main()
    int-list: _3_a, _3_b, _t1, _t2, _t3
    float-list:
    main:
        assign, _3_a, 10,
        assign, _3_b, 20,
        mult, _3_a, 2, _t1
        sub, _3_b, _3_a, _t2
        add, _3_a, _3_b, _t3
        call, printout, _t1, _t2, _t3
        return, 0, ,
end_function main

end_program demo_print
```

demo_slope.ir

```
start_program demo_slope
    static-int-list:
    static-float-list: _0_p1[2], _0_p2[2], _0_rise, _0_run,
       _0_slope, _0_yintercept, _0_xintercept

start_function main
    int main()
    int-list: _t7, _t8, _t9
    float-list: _t1, _t2, _t3, _t4, _t5, _t6, _t10, _t11,
            _t12, _t13, _t14, _t15, _t16
    main:
        array_store, _0_p1, 0, 2.0
        array_store, _0_p1, 1, 1.0
        array_store, _0_p2, 0, 3.0
        array_store, _0_p2, 1, 3.0
        array_load, _t1, _0_p2, 1
        array_load, _t2, _0_p1, 1
        sub, _t1, _t2, _t3
        assign, _0_rise, _t3,
        array_load, _t4, _0_p2, 0
        array_load, _t5, _0_p1, 0
        sub, _t4, _t5, _t6
        assign, _0_run, _t6,
        assign, _t7, 1,
        breq, _0_run, 0.0, _L1
        assign, _t7, 0,
    _L1:
        assign, _t8, 1,
        breq, _0_rise, 0.0, _L2
        assign, _t8, 0,
    _L2:
        or, _t7, _t8, _t9
        breq, _t9, 0, _L0
        call, printi, 0
        call, exit, 1
    _L0:
        div, _0_rise, _0_run, _t10
        assign, _0_slope, _t10,
        array_load, _t11, _0_p2, 1
        array_load, _t12, _0_p2, 0
        mult, _0_slope, _t12, _t13
        sub, _t11, _t13, _t14
        assign, _0_yintercept, _t14,
        sub, 0, _0_yintercept, _t15
        div, _t15, _0_slope, _t16
        assign, _0_xintercept, _t16,
        call, printf, _0_slope
        call, printf, _0_xintercept
        call, printf, _0_yintercept
        return, 0, ,
end_function main

end_program demo_slope
```

demo_selection_sort.ir

```
start_program demo_selection_sort
    static-int-list:
    static-float-list:

start_function main
    int main()
    int-list: _2_cats[8], _2_index, _2_lowestIndex,
       _2_subIndex, _2_size, _2_smallest, _2_first, _t1,
       _t2, _t3, _t4, _t5, _t6, _t7, _t8, _t9, _t10
    float-list:
    main:
        array_store, _2_cats, 0, 7
        array_store, _2_cats, 1, 2
        array_store, _2_cats, 2, 14
        array_store, _2_cats, 3, 24
        array_store, _2_cats, 4, 5
        array_store, _2_cats, 5, 6
        array_store, _2_cats, 6, 49
        array_store, _2_cats, 7, 33
        assign, _2_size, 8,
        assign, _2_index, 0,
    _L0:
        sub, _2_size, 1, _t1
        brgt, _2_index, _t1, _L1
        assign, _2_lowestIndex, _2_index,
        add, _2_index, 1, _t2
        assign, _2_subIndex, _t2,
    _L2:
        sub, _2_size, 1, _t3
        brgt, _2_subIndex, _t3, _L3
        array_load, _t4, _2_cats, _2_subIndex
        array_load, _t5, _2_cats, _2_lowestIndex
        assign, _t6, 1,
        brlt, _t4, _t5, _L5
        assign, _t6, 0,
    _L5:
        breq, _t6, 0, _L4
        assign, _2_lowestIndex, _2_subIndex,
    _L4:
        add, _2_subIndex, 1, _2_subIndex
        goto, _L2, ,
    _L3:
        array_load, _t7, _2_cats, _2_index
        assign, _2_first, _t7,
        array_load, _t8, _2_cats, _2_lowestIndex
        assign, _2_smallest, _t8,
        array_store, _2_cats, _2_index, _2_smallest
        array_store, _2_cats, _2_lowestIndex, _2_first
        add, _2_index, 1, _2_index
        goto, _L0, ,
```

demo_selection_sort.ir - continued

```
     _L1:
         assign, _2_index, 0,
     _L6:
         sub, _2_size, 1, _t9
         brgt, _2_index, _t9, _L7
         array_load, _t10, _2_cats, _2_index
         call, printi, _t10
         add, _2_index, 1, _2_index
         goto, _L6, ,
     _L7:
         return, 0, ,
 end_function main

 end_program demo_selection_sort
```